

语言教学预约系统-2.3学生预约模块+2.4课程执行与后续流程模块API实现

说明：本文件独立实现设计说明中2.3学生预约模块、2.4课程执行与后续流程模块的所有API，采用Spring Boot + MyBatis架构，遵循原有权限控制、数据校验、统一返回格式规范，可直接集成到原有项目中，所有代码标注对应设计项目。

一、2.3 学生预约模块API实现（对应设计2.3 学生预约模块）

1.1 核心实体类（关联原有实体，新增专属实体）

```
1  package com.language.reservation.entity;
2
3  import lombok.Data;
4  import javax.validation.constraints.NotBlank;
5  import javax.validation.constraints.NotNull;
6  import java.math.BigDecimal;
7  import java.util.Date;
8
9  /**
10   * 预约订单实体类，对应设计2.3 学生预约模块-预约下单、支付、取消等接口
11   * 关联排期 (scheduleId)、学生 (studentId)、课程 (courseId)
12   */
13  @Data
14  public class ReservationOrder {
15      private String orderId; // 唯一标识 (UUID)，对应通用校验规则-ID类参数
16      @NotBlank(message = "排期ID不能为空")
17      private String scheduleId; // 关联课程排期
18      @NotBlank(message = "课程ID不能为空")
19      private String courseId; // 关联教师课程
20      @NotBlank(message = "学生ID不能为空")
21      private String studentId; // 关联学生
22      @NotNull(message = "订单金额不能为空")
23      private BigDecimal orderAmount; // 订单金额 (对应课程模板课时费)，保留2位小数
24      @NotBlank(message = "订单状态不能为空")
25      private String orderStatus; // 枚举值: pending (待支付) /paid (已支付) /cancelled (已取消) /refunded (已退款)
26      private String payTime; // 支付时间，格式YYYY-MM-DD HH:mm:ss (对应通用校验规则-时间)
```

```

27     private String cancelTime; // 取消时间, 格式同上
28     private String refundTime; // 退款时间, 格式同上
29     private String payNo;      // 支付流水号 (支付宝返回)
30     private Date createTime;   // 订单创建时间 (自动生成)
31 }
32
33 /**
34  * 预约查询条件实体, 对应设计2.3 预约查询接口的请求参数
35  */
36 @Data
37 public class ReservationQuery {
38     private String studentId; // 学生ID (默认当前登录学生)
39     private String courseId;  // 课程ID (可选)
40     private String scheduleId; // 排期ID (可选)
41     private String orderStatus; // 订单状态 (可选, 筛选对应状态订单)
42     private String startDate; // 开始日期 (可选, 格式YYYY-MM-DD)
43     private String endDate;   // 结束日期 (可选, 格式YYYY-MM-DD)
44 }
45

```

1.2 控制器 (StudentReservationController.java)

```

1  package com.language.reservation.controller;
2
3  import com.language.reservation.common.Result;
4  import com.language.reservation.entity.ReservationOrder;
5  import com.language.reservation.entity.ReservationQuery;
6  import com.language.reservation.service.StudentReservationService;
7  import com.language.reservation.utils.PermissionCheck;
8  import org.springframework.beans.factory.annotation.Autowired;
9  import org.springframework.validation.annotation.Validated;
10 import org.springframework.web.bind.annotation.*;
11
12 import javax.servlet.http.HttpServletRequest;
13 import java.util.List;
14 import java.util.Map;
15
16 /**
17  * 学生预约模块控制器, 对应设计2.3 学生预约模块所有接口
18  * 接口权限: 仅学生可操作 (通过PermissionCheck校验)
19  */
20 @RestController
21 @RequestMapping("/reservation")
22 @Validated
23 public class StudentReservationController {

```

```
24
25     @Autowired
26     private StudentReservationService reservationService;
27     @Autowired
28     private PermissionCheck permissionCheck;
29
30     /**
31      * 预约下单接口, 对应设计2.3 接口: /api/v1/reservation/create (学生权限)
32      * 功能: 学生选择排期创建预约订单, 生成待支付订单
33      */
34     @PostMapping("/create")
35     public Result<Map<String, String>> createReservation(
36         @Validated @RequestBody ReservationOrder order,
37         @RequestHeader("Authorization") String token) {
38         // 权限校验: 仅学生可操作
39         permissionCheck.checkStudent(token);
40         // 校验订单归属当前学生 (确保学生只能创建自己的订单)
41         String studentId = permissionCheck.getUserIdFromToken(token);
42         order.setStudentId(studentId);
43         // 调用服务层创建订单
44         Map<String, String> resultMap =
45             reservationService.createReservation(order);
46         return Result.success(resultMap, "预约订单创建成功, 请尽快支付");
47     }
48
49     /**
50      * 订单支付接口, 对应设计2.3 接口: /api/v1/reservation/pay (学生权限)
51      * 功能: 学生支付待支付订单, 对接支付宝支付
52      */
53     @PostMapping("/pay")
54     public Result<Map<String, String>> payOrder(
55         @NotBlank(message = "订单ID不能为空") String orderId,
56         @RequestHeader("Authorization") String token) {
57         // 权限校验: 仅学生可操作
58         permissionCheck.checkStudent(token);
59         // 校验订单归属当前学生
60         String studentId = permissionCheck.getUserIdFromToken(token);
61         permissionCheck.checkStudentOwner(token, studentId);
62         // 调用服务层发起支付, 返回支付宝支付页面链接
63         Map<String, String> payInfo = reservationService.payOrder(orderId,
64             studentId);
65         return Result.success(payInfo, "支付链接生成成功");
66     }
67
68     /**
69      * 支付同步回调接口, 对应设计2.3 支付功能-同步回调
70      * 功能: 支付宝支付成功后跳转回调, 更新订单状态
```

```

69     */
70     @GetMapping("/pay/return")
71     public Result<Map<String, String>> payReturn(HttpServletRequest request) {
72         // 解析支付宝回调参数, 更新订单状态
73         Map<String, String> resultMap =
reservationService.handlePayReturn(request);
74         return Result.success(resultMap, "支付成功");
75     }
76
77     /**
78      * 支付异步回调接口, 对应设计2.3 支付功能-异步回调
79      * 功能: 支付宝后台异步通知, 确保订单状态同步 (防止前端未回调)
80      */
81     @PostMapping("/pay/notify")
82     public String payNotify(HttpServletRequest request) {
83         // 处理支付宝异步通知, 更新订单状态, 返回success告知支付宝已接收
84         boolean handleSuccess = reservationService.handlePayNotify(request);
85         return handleSuccess ? "success" : "fail";
86     }
87
88     /**
89      * 取消预约接口, 对应设计2.3 接口: /api/v1/reservation/cancel (学生权限)
90      * 功能: 学生取消未支付订单, 或符合条件的已支付订单 (未到课)
91      */
92     @PostMapping("/cancel")
93     public Result<Void> cancelReservation(
94         @NotBlank(message = "订单ID不能为空") String orderId,
95         @RequestHeader("Authorization") String token) {
96         // 权限校验: 仅学生可操作
97         permissionCheck.checkStudent(token);
98         // 校验订单归属当前学生
99         String studentId = permissionCheck.getUserIdFromToken(token);
100        reservationService.checkOrderOwner(orderId, studentId);
101        // 调用服务层取消订单
102        reservationService.cancelReservation(orderId);
103        return Result.success(null, "预约订单取消成功");
104    }
105
106    /**
107     * 预约查询接口, 对应设计2.3 接口: /api/v1/reservation/list (学生权限)
108     * 功能: 学生查询自己的所有预约订单, 支持多条件筛选
109     */
110    @GetMapping("/list")
111    public Result<Map<String, Object>> getReservationList(
112        ReservationQuery query,
113        @RequestHeader("Authorization") String token) {
114        // 权限校验: 仅学生可操作

```

```

115         permissionCheck.checkStudent(token);
116         // 设置当前学生ID为查询条件 (默认查询自己的订单)
117         String studentId = permissionCheck.getUserIdFromToken(token);
118         query.setStudentId(studentId);
119         // 调用服务层查询订单列表 (含分页)
120         Map<String, Object> resultMap =
reservationService.getReservationList(query);
121         return Result.success(resultMap, "订单查询成功");
122     }
123
124     /**
125      * 订单详情接口, 对应设计2.3 接口: /api/v1/reservation/detail (学生权限)
126      * 功能: 学生查看单个预约订单的详细信息
127      */
128     @GetMapping("/detail")
129     public Result<ReservationOrder> getReservationDetail(
130         @NotBlank(message = "订单ID不能为空") String orderId,
131         @RequestHeader("Authorization") String token) {
132         // 权限校验: 仅学生可操作
133         permissionCheck.checkStudent(token);
134         // 校验订单归属当前学生
135         String studentId = permissionCheck.getUserIdFromToken(token);
136         reservationService.checkOrderOwner(orderId, studentId);
137         // 调用服务层查询订单详情
138         ReservationOrder order =
reservationService.getReservationDetail(orderId);
139         return Result.success(order, "订单详情查询成功");
140     }
141 }
142

```

1.3 服务层 (StudentReservationService.java)

```

1  package com.language.reservation.service;
2
3  import com.alipay.api.AlipayApiException;
4  import com.alipay.api.AlipayClient;
5  import com.alipay.api.DefaultAlipayClient;
6  import com.alipay.api.request.AlipayTradePagePayRequest;
7  import com.language.reservation.entity.Course;
8  import com.language.reservation.entity.ReservationOrder;
9  import com.language.reservation.entity.ReservationQuery;
10 import com.language.reservation.entity.Schedule;
11 import com.language.reservation.exception.BusinessException;
12 import com.language.reservation.exception.ResourceNotFoundException;

```

```
13 import com.language.reservation.mapper.CourseMapper;
14 import com.language.reservation.mapper.ReservationOrderMapper;
15 import com.language.reservation.mapper.ScheduleMapper;
16 import org.springframework.beans.factory.annotation.Autowired;
17 import org.springframework.beans.factory.annotation.Value;
18 import org.springframework.stereotype.Service;
19 import org.springframework.transaction.annotation.Transactional;
20
21 import javax.servlet.http.HttpServletRequest;
22 import java.text.ParseException;
23 import java.text.SimpleDateFormat;
24 import java.util.Date;
25 import java.util.HashMap;
26 import java.util.List;
27 import java.util.Map;
28 import java.util.UUID;
29
30 /**
31  * 学生预约模块服务层，对应设计2.3 学生预约模块所有接口的业务逻辑
32  * 涵盖预约下单、支付、取消、查询等功能，严格遵循权限校验和业务规则
33  */
34 @Service
35 public class StudentReservationService {
36
37     @Autowired
38     private ReservationOrderMapper orderMapper;
39     @Autowired
40     private ScheduleMapper scheduleMapper;
41     @Autowired
42     private CourseMapper courseMapper;
43
44     // 支付宝配置 (从application.yml读取，对应设计2.3 支付功能)
45     @Value("${alipay.app-id}")
46     private String alipayAppId;
47     @Value("${alipay.private-key}")
48     private String alipayPrivateKey;
49     @Value("${alipay.public-key}")
50     private String alipayPublicKey;
51     @Value("${alipay.gateway-url}")
52     private String alipayGatewayUrl;
53     @Value("${alipay.return-url}")
54     private String alipayReturnUrl;
55     @Value("${alipay.notify-url}")
56     private String alipayNotifyUrl;
57
58     // 日期格式化工具，对应通用校验规则-时间格式
```

```

59     private static final SimpleDateFormat DATE_FORMAT = new
SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
60
61     /**
62      * 预约下单, 对应设计2.3 预约下单接口
63      */
64     @Transactional
65     public Map<String, String> createReservation(ReservationOrder order) {
66         // 1. 校验排期是否存在且可预约
67         Schedule schedule =
scheduleMapper.selectScheduleById(order.getScheduleId());
68         if (schedule == null) {
69             throw new ResourceNotFoundException("排期不存在, 无法预约");
70         }
71         if (!"available".equals(schedule.getStatus())) {
72             throw new BusinessException("该排期不可预约 (已被占用或已取消) ");
73         }
74
75         // 2. 校验课程是否存在
76         Course course = courseMapper.selectCourseById(order.getCourseId());
77         if (course == null) {
78             throw new ResourceNotFoundException("课程不存在, 无法预约");
79         }
80
81         // 3. 校验排期与课程关联关系 (防止非法预约)
82         if (!schedule.getCourseId().equals(order.getCourseId())) {
83             throw new BusinessException("排期与课程不匹配, 无法预约");
84         }
85
86         // 4. 校验学生是否已预约该排期 (避免重复预约, 对应设计2.3 预约规则)
87         if (orderMapper.selectByStudentAndSchedule(order.getStudentId(),
order.getScheduleId()) != null) {
88             throw new BusinessException("您已预约该排期, 请勿重复预约");
89         }
90
91         // 5. 生成订单ID, 设置订单默认状态 (待支付)
92         String orderId = UUID.randomUUID().toString();
93         order.setOrderId(orderId);
94         order.setOrderStatus("pending");
95         order.setCreateTime(new Date());
96
97         // 6. 插入订单到数据库
98         orderMapper.insertOrder(order);
99
100        // 7. 组装返回数据 (订单ID、支付提示)
101        Map<String, String> resultMap = new HashMap<>();
102        resultMap.put("orderId", orderId);

```

```

103         resultMap.put("tips", "订单创建成功, 30分钟内未支付将自动取消");
104         return resultMap;
105     }
106
107     /**
108      * 订单支付, 对接支付宝, 对应设计2.3 支付功能
109      */
110     public Map<String, String> payOrder(String orderId, String studentId) {
111         // 1. 校验订单是否存在
112         ReservationOrder order = orderMapper.selectOrderById(orderId);
113         if (order == null) {
114             throw new ResourceNotFoundException("订单不存在, 无法支付");
115         }
116
117         // 2. 校验订单归属当前学生
118         if (!order.getStudentId().equals(studentId)) {
119             throw new BusinessException("您无权支付他人订单");
120         }
121
122         // 3. 校验订单状态 (仅待支付订单可支付)
123         if (!"pending".equals(order.getOrderStatus())) {
124             throw new BusinessException("订单状态异常, 仅待支付订单可支付");
125         }
126
127         // 4. 校验订单支付超时 (30分钟超时, 对应设计2.3 预约规则)
128         long createTime = order.getCreateTime().getTime();
129         long currentTime = System.currentTimeMillis();
130         if (currentTime - createTime > 30 * 60 * 1000) {
131             // 超时自动取消订单
132             order.setOrderStatus("cancelled");
133             order.setCancelTime(DateFormat.format(new Date()));
134             orderMapper.updateOrderStatus(order);
135             throw new BusinessException("订单已超时, 已自动取消, 请重新预约");
136         }
137
138         // 5. 对接支付宝, 生成支付链接
139         AlipayClient alipayClient = new DefaultAlipayClient(
140             alipayGatewayUrl, alipayAppId, alipayPrivateKey, "json", "UTF-
8", alipayPublicKey, "RSA2");
141         AlipayTradePagePayRequest request = new AlipayTradePagePayRequest();
142         // 设置回调地址
143         request.setReturnUrl(alipayReturnUrl);
144         request.setNotifyUrl(alipayNotifyUrl);
145
146         // 组装支付参数
147         Map<String, String> bizContent = new HashMap<>();
148         bizContent.put("out_trade_no", orderId); // 订单号 (系统生成)

```



```

149         bizContent.put("total_amount", order.getOrderAmount().toString()); //
订单金额
150         bizContent.put("subject", "语言教学预约-" + order.getCourseId()); // 订
单标题
151         bizContent.put("product_code", "FAST_INSTANT_TRADE_PAY"); // 支付宝产品
码
152
153         request.setBizContent(new
com.alibaba.fastjson.JSONObject(bizContent).toString());
154
155         try {
156             // 调用支付宝接口, 获取支付页面链接
157             String form = alipayClient.pageExecute(request).getBody();
158             Map<String, String> resultMap = new HashMap<>();
159             resultMap.put("payForm", form); // 支付表单 (前端渲染即可跳转支付宝)
160             return resultMap;
161         } catch (AlipayApiException e) {
162             e.printStackTrace();
163             throw new BusinessException("支付链接生成失败, 请稍后再试");
164         }
165     }
166
167     /**
168      * 支付同步回调处理, 对应设计2.3 支付功能-同步回调
169      */
170     @Transactional
171     public Map<String, String> handlePayReturn(HttpServletRequest request) {
172         // 1. 解析支付宝回调参数 (简化处理, 实际需校验签名)
173         String orderId = request.getParameter("out_trade_no");
174         String tradeNo = request.getParameter("trade_no"); // 支付宝流水号
175         String tradeStatus = request.getParameter("trade_status"); // 支付状态
176
177         // 2. 校验支付状态 (TRADE_SUCCESS为支付成功)
178         if (!"TRADE_SUCCESS".equals(tradeStatus)) {
179             throw new BusinessException("支付未成功, 请核对支付状态");
180         }
181
182         // 3. 校验订单是否存在
183         ReservationOrder order = orderMapper.selectOrderById(orderId);
184         if (order == null) {
185             throw new ResourceNotFoundException("订单不存在, 支付回调失败");
186         }
187
188         // 4. 更新订单状态为已支付
189         order.setOrderStatus("paid");
190         order.setPayTime(DateFormat.format(new Date()));
191         order.setPayNo(tradeNo);

```

```

192         orderMapper.updateOrderStatus(order);
193
194         // 5. 更新排期状态为不可预约（避免重复预约）
195         Schedule schedule =
scheduleMapper.selectScheduleById(order.getScheduleId());
196         schedule.setStatus("unavailable");
197         scheduleMapper.updateScheduleStatus(schedule);
198
199         // 6. 组装返回数据
200         Map<String, String> resultMap = new HashMap<>();
201         resultMap.put("orderId", orderId);
202         resultMap.put("tradeNo", tradeNo);
203         resultMap.put("message", "支付成功, 您的预约已确认");
204         return resultMap;
205     }
206
207     /**
208      * 支付异步回调处理, 对应设计2.3 支付功能-异步回调
209      */
210     @Transactional
211     public boolean handlePayNotify(HttpServletRequest request) {
212         try {
213             // 1. 解析支付宝回调参数（简化处理，实际需校验签名）
214             String orderId = request.getParameter("out_trade_no");
215             String tradeNo = request.getParameter("trade_no");
216             String tradeStatus = request.getParameter("trade_status");
217
218             // 2. 校验支付状态和订单
219             if (!"TRADE_SUCCESS".equals(tradeStatus)) {
220                 return false;
221             }
222             ReservationOrder order = orderMapper.selectOrderById(orderId);
223             if (order == null || !"pending".equals(order.getOrderStatus())) {
224                 return false;
225             }
226
227             // 3. 更新订单状态（与同步回调逻辑一致，防止前端未回调）
228             order.setOrderStatus("paid");
229             order.setPayTime(DateFormat.format(new Date()));
230             order.setPayNo(tradeNo);
231             orderMapper.updateOrderStatus(order);
232
233             // 4. 更新排期状态
234             Schedule schedule =
scheduleMapper.selectScheduleById(order.getScheduleId());
235             schedule.setStatus("unavailable");
236             scheduleMapper.updateScheduleStatus(schedule);

```

```
237
238         return true;
239     } catch (Exception e) {
240         e.printStackTrace();
241         return false;
242     }
243 }
244
245 /**
246  * 取消预约订单，对应设计2.3 取消预约接口
247  */
248 @Transactional
249 public void cancelReservation(String orderId) {
250     // 1. 校验订单是否存在
251     ReservationOrder order = orderMapper.selectOrderById(orderId);
252     if (order == null) {
253         throw new ResourceNotFoundException("订单不存在，无法取消");
254     }
255
256     // 2. 校验订单状态（仅待支付、已支付且未到课的订单可取消）
257     if ("cancelled".equals(order.getOrderStatus()) ||
258 "refunded".equals(order.getOrderStatus())) {
259         throw new BusinessException("订单已取消或已退款，无需重复操作");
260     }
261
262     // 3. 已支付订单取消：校验是否已到课（到课不可取消）
263     if ("paid".equals(order.getOrderStatus())) {
264         Schedule schedule =
265 scheduleMapper.selectScheduleById(order.getScheduleId());
266         try {
267             Date startTime = DATE_FORMAT.parse(schedule.getStartTime());
268             Date currentTime = new Date();
269             if (currentTime.after(startTime)) {
270                 throw new BusinessException("课程已开始，无法取消预约");
271             }
272             // 已支付取消，后续可添加退款逻辑（对应设计2.3 退款规则）
273             order.setOrderStatus("refunded");
274             order.setRefundTime(DATE_FORMAT.format(new Date()));
275             // 恢复排期可预约状态
276             schedule.setStatus("available");
277             scheduleMapper.updateScheduleStatus(schedule);
278         } catch (ParseException e) {
279             throw new BusinessException("时间格式错误，无法取消预约");
280         }
281     } else {
282         // 待支付订单取消
283         order.setOrderStatus("cancelled");
284     }
285 }
```

```
282     }
283
284     // 4. 更新订单取消时间和状态
285     order.setCancelTime(DATE_FORMAT.format(new Date()));
286     orderMapper.updateOrderStatus(order);
287 }
288
289 /**
290  * 查询预约订单列表，对应设计2.3 预约查询接口
291  */
292 public Map<String, Object> getReservationList(ReservationQuery query) {
293     // 1. 查询订单列表（支持多条件筛选）
294     List<ReservationOrder> orderList = orderMapper.selectOrderList(query);
295     // 2. 查询总条数（分页用）
296     int total = orderMapper.selectOrderCount(query);
297     // 3. 组装返回数据（列表+总条数）
298     Map<String, Object> resultMap = new HashMap<>();
299     resultMap.put("orderList", orderList);
300     resultMap.put("total", total);
301     return resultMap;
302 }
303
304 /**
305  * 查询订单详情，对应设计2.3 订单详情接口
306  */
307 public ReservationOrder getReservationDetail(String orderId) {
308     ReservationOrder order = orderMapper.selectOrderById(orderId);
309     if (order == null) {
310         throw new ResourceNotFoundException("订单不存在");
311     }
312     return order;
313 }
314
315 /**
316  * 校验订单归属（辅助方法），对应设计2.3 权限控制
317  */
318 public void checkOrderOwner(String orderId, String studentId) {
319     ReservationOrder order = orderMapper.selectOrderById(orderId);
320     if (order == null) {
321         throw new ResourceNotFoundException("订单不存在");
322     }
323     if (!order.getStudentId().equals(studentId)) {
324         throw new BusinessException("您无权操作他人订单");
325     }
326 }
327 }
328
```

1.4 Mapper接口（核心方法，对应数据库操作）

```
1  package com.language.reservation.mapper;
2
3  import com.language.reservation.entity.ReservationOrder;
4  import com.language.reservation.entity.ReservationQuery;
5  import org.apache.ibatis.annotations.Mapper;
6  import org.apache.ibatis.annotations.Param;
7
8  import java.util.List;
9
10 /**
11  * 预约订单Mapper接口，对应数据库操作
12  */
13 @Mapper
14 public interface ReservationOrderMapper {
15     // 插入订单
16     int insertOrder(ReservationOrder order);
17
18     // 根据订单ID查询订单
19     ReservationOrder selectOrderById(String orderId);
20
21     // 根据学生ID和排期ID查询订单（校验重复预约）
22     ReservationOrder selectByStudentAndSchedule(@Param("studentId") String
studentId, @Param("scheduleId") String scheduleId);
23
24     // 更新订单状态（支付、取消、退款）
25     int updateOrderStatus(ReservationOrder order);
26
27     // 多条件查询订单列表
28     List<ReservationOrder> selectOrderList(ReservationQuery query);
29
30     // 多条件查询订单总条数
31     int selectOrderCount(ReservationQuery query);
32 }
33
34 // ScheduleMapper补充方法（原有基础上新增）
35 package com.language.reservation.mapper;
36
37 import com.language.reservation.entity.Schedule;
38 import org.apache.ibatis.annotations.Mapper;
39 import org.apache.ibatis.annotations.Param;
40
41 @Mapper
```

```

42 public interface ScheduleMapper {
43     // 原有方法省略...
44
45     // 更新排期状态 (可预约/不可预约)
46     int updateScheduleStatus(Schedule schedule);
47
48     // 根据课程ID和时间查询排期 (避免排期冲突)
49     Schedule selectScheduleByTime(@Param("courseId") String courseId,
50     @Param("startTime") String startTime, @Param("endTime") String endTime);
51 }

```

二、2.4 课程执行与后续流程模块API实现（对应设计2.4 课程执行与后续流程模块）

2.1 核心实体类（新增专属实体，关联原有实体）

```

1  package com.language.reservation.entity;
2
3  import lombok.Data;
4  import javax.validation.constraints.NotBlank;
5  import javax.validation.constraints.NotNull;
6  import javax.validation.constraints.Size;
7
8  /**
9   * 课程签到实体类，对应设计2.4 课程执行-签到功能
10  */
11  @Data
12  public class CourseCheckIn {
13      private String checkInId; // 唯一标识 (UUID)
14      @NotBlank(message = "订单ID不能为空")
15      private String orderId; // 关联预约订单
16      @NotBlank(message = "排期ID不能为空")
17      private String scheduleId; // 关联课程排期
18      @NotBlank(message = "学生ID不能为空")
19      private String studentId; // 关联学生
20      @NotBlank(message = "教师ID不能为空")
21      private String teacherId; // 关联教师
22      @NotBlank(message = "签到状态不能为空")
23      private String checkInStatus; // 枚举值: checked (已签到) /unchecked (未签到)
24      private String checkInTime; // 签到时间, 格式YYYY-MM-DD HH:mm:ss
25  }

```

```

26
27 /**
28  * 课程评价实体类，对应设计2.4 后续流程-课程评价功能
29  */
30 @Data
31 public class CourseEvaluation {
32     private String evaluationId; // 唯一标识 (UUID)
33     @NotBlank(message = "订单ID不能为空")
34     private String orderId; // 关联预约订单 (课程结束后可评价)
35     @NotBlank(message = "学生ID不能为空")
36     private String studentId; // 关联评价学生
37     @NotBlank(message = "教师ID不能为空")
38     private String teacherId; // 关联被评价教师
39     @NotNull(message = "评价分数不能为空")
40     private Integer score; // 评价分数 (1-5分，对应设计2.4 评价规则)
41     @NotBlank(message = "评价内容不能为空")
42     @Size(min = 10, max = 500, message = "评价内容需10-500字")
43     private String content; // 评价内容
44     private String createTime; // 评价时间，格式YYYY-MM-DD HH:mm:ss
45 }
46
47 /**
48  * 课程反馈实体类，对应设计2.4 后续流程-课程反馈功能
49  */
50 @Data
51 public class CourseFeedback {
52     private String feedbackId; // 唯一标识 (UUID)
53     @NotBlank(message = "订单ID不能为空")
54     private String orderId; // 关联预约订单
55     @NotBlank(message = "提交人ID不能为空")
56     private String submitterId; // 提交人ID (学生/教师)
57     @NotBlank(message = "提交人角色不能为空")
58     private String submitterRole; // 提交人角色 (student/teacher)
59     @NotBlank(message = "反馈类型不能为空")
60     private String feedbackType; // 枚举值: content (内容反馈) /service (服务反
        馈) /other (其他反馈)
61     @NotBlank(message = "反馈内容不能为空")
62     @Size(min = 10, max = 500, message = "反馈内容需10-500字")
63     private String content; // 反馈内容
64     private String createTime; // 反馈时间，格式YYYY-MM-DD HH:mm:ss
65     private String handleStatus; // 处理状态: pending (待处理) /handled (已处理)
66     private String handleContent; // 处理内容 (管理员填写)
67     private String handleTime; // 处理时间
68 }
69

```

2.2 控制器 (CourseExecutionController.java)

```
1  package com.language.reservation.controller;
2
3  import com.language.reservation.common.Result;
4  import com.language.reservation.entity.CourseCheckIn;
5  import com.language.reservation.entity.CourseEvaluation;
6  import com.language.reservation.entity.CourseFeedback;
7  import com.language.reservation.service.CourseExecutionService;
8  import com.language.reservation.utils.PermissionCheck;
9  import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.validation.annotation.Validated;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14 import java.util.Map;
15
16 /**
17  * 课程执行与后续流程控制器，对应设计2.4 课程执行与后续流程模块所有接口
18  * 接口权限：学生、教师、管理员分别对应不同操作权限
19  */
20 @RestController
21 @RequestMapping("/course/execution")
22 @Validated
23 public class CourseExecutionController {
24
25     @Autowired
26     private CourseExecutionService executionService;
27     @Autowired
28     private PermissionCheck permissionCheck;
29
30     /**
31      * 学生签到接口，对应设计2.4 接口：/api/v1/course/execution/checkIn (学生权限)
32      * 功能：学生在课程开始后进行签到
33      */
34     @PostMapping("/checkIn")
35     public Result<Void> studentCheckIn(
36         @Validated @RequestBody CourseCheckIn checkIn,
37         @RequestHeader("Authorization") String token) {
38         // 权限校验：仅学生可操作
39         permissionCheck.checkStudent(token);
40         // 校验签到归属当前学生
41         String studentId = permissionCheck.getUserIdFromToken(token);
42         if (!studentId.equals(checkIn.getStudentId())) {
43             throw new BusinessException("您无权为他人签到");
44         }
45     }
46 }
```



```

45         // 调用服务层完成签到
46         executionService.studentCheckIn(checkIn);
47         return Result.success(null, "签到成功");
48     }
49
50     /**
51      * 教师查询签到列表接口, 对应设计2.4 接
52      □: /api/v1/course/execution/checkIn/list (教师权限)
53      * 功能: 教师查询自己课程的学生签到情况
54      */
55     @GetMapping("/checkIn/list")
56     public Result<List<CourseCheckIn>> getCheckInList(
57         @NotBlank(message = "排期ID不能为空") String scheduleId,
58         @RequestHeader("Authorization") String token) {
59         // 权限校验: 仅教师可操作
60         permissionCheck.checkTeacher(token);
61         // 校验排期归属当前教师
62         String teacherId = permissionCheck.getUserIdFromToken(token);
63         executionService.checkScheduleOwner(scheduleId, teacherId);
64         // 调用服务层查询签到列表
65         List<CourseCheckIn> checkInList =
66         executionService.getCheckInList(scheduleId);
67         return Result.success(checkInList, "签到列表查询成功");
68     }
69
70     /**
71      * 学生提交课程评价接口, 对应设计2.4 接
72      □: /api/v1/course/execution/evaluation/add (学生权限)
73      * 功能: 学生在课程结束后提交评价
74      */
75     @PostMapping("/evaluation/add")
76     public Result<Map<String, String>> addEvaluation(
77         @Validated @RequestBody CourseEvaluation evaluation,
78         @RequestHeader("Authorization") String token) {
79         // 权限校验: 仅学生可操作
80         permissionCheck.checkStudent(token);
81         // 校验评价归属当前学生
82         String studentId = permissionCheck.getUserIdFromToken(token);
83         if (!studentId.equals(evaluation.getStudentId())) {
84             throw new BusinessException("您无权为他人提交评价");
85         }
86         // 调用服务层提交评价
87         Map<String, String> resultMap =
88         executionService.addEvaluation(evaluation);
89         return Result.success(resultMap, "评价提交成功, 感谢您的反馈");
90     }

```

```

88      /**
89       * 教师查询评价列表接口，对应设计2.4 接
□: /api/v1/course/execution/evaluation/list (教师权限)
90       * 功能：教师查询自己收到的所有课程评价
91       */
92     @GetMapping("/evaluation/list")
93     public Result<List<CourseEvaluation>> getEvaluationList(
94         @RequestHeader("Authorization") String token) {
95         // 权限校验：仅教师可操作
96         permissionCheck.checkTeacher(token);
97         // 获取当前教师ID
98         String teacherId = permissionCheck.getUserIdFromToken(token);
99         // 调用服务层查询评价列表
100        List<CourseEvaluation> evaluationList =
executionService.getEvaluationList(teacherId);
101        return Result.success(evaluationList, "评价列表查询成功");
102    }
103
104    /**
105     * 提交课程反馈接口，对应设计2.4 接口： /api/v1/course/execution/feedback/add
(学生/教师权限)
106     * 功能：学生或教师提交课程相关反馈
107     */
108     @PostMapping("/feedback/add")
109     public Result<Map<String, String>> addFeedback(
110         @Validated @RequestBody CourseFeedback feedback,
111         @RequestHeader("Authorization") String token) {
112         // 权限校验：学生或教师可操作
113         String role = permissionCheck.jwtUtil.getRoleFromToken(token);
114         if (!"student".equals(role) && !"teacher".equals(role)) {
115             throw new BusinessException("仅学生和教师可提交反馈");
116         }
117         // 校验反馈归属当前提交人
118         String submitterId = permissionCheck.getUserIdFromToken(token);
119         if (!submitterId.equals(feedback.getSubmitterId())) {
120             throw new BusinessException("您无权提交他人反馈");
121         }
122         // 调用服务层提交反馈
123         Map<String, String> resultMap =
executionService.addFeedback(feedback);
124         return Result.success(resultMap, "反馈提交成功，我们将尽快处理");
125     }
126
127    /**
128     * 管理员处理反馈接口，对应设计2.4 接
□: /api/v1/course/execution/feedback/handle (管理员权限)
129     * 功能：管理员处理学生/教师提交的反馈

```

```

130     */
131     @PostMapping("/feedback/handle")
132     public Result<Void> handleFeedback(
133         @NotBlank(message = "反馈ID不能为空") String feedbackId,
134         @NotBlank(message = "处理内容不能为空") String handleContent,
135         @RequestHeader("Authorization") String token) {
136         // 权限校验：仅管理员可操作
137         permissionCheck.checkAdmin(token);
138         // 调用服务层处理反馈
139         executionService.handleFeedback(feedbackId, handleContent);
140         return Result.success(null, "反馈处理成功");
141     }
142
143     /**
144      * 管理员查询反馈列表接口，对应设计2.4 接
145      □: /api/v1/course/execution/feedback/list (管理员权限)
146      * 功能：管理员查询所有反馈，支持按处理状态筛选
147      */
148     @GetMapping("/feedback/list")
149     public Result<List<CourseFeedback>> getFeedbackList(
150         String handleStatus,
151         @RequestHeader("Authorization") String token) {
152         // 权限校验：仅管理员可操作
153         permissionCheck.checkAdmin(token);
154         // 调用服务层查询反馈列表
155         List<CourseFeedback> feedbackList =
156         executionService.getFeedbackList(handleStatus);
157         return Result.success(feedbackList, "反馈列表查询成功");
158     }
159 }

```

2.3 服务层 (CourseExecutionService.java)

```

1  package com.language.reservation.service;
2
3  import com.language.reservation.entity.*;
4  import com.language.reservation.exception.BusinessException;
5  import com.language.reservation.exception.ResourceNotFoundException;
6  import com.language.reservation.mapper.*;
7  import org.springframework.beans.factory.annotation.Autowired;
8  import org.springframework.stereotype.Service;
9  import org.springframework.transaction.annotation.Transactional;
10
11  import java.text.ParseException;

```

```
12 import java.text.SimpleDateFormat;
13 import java.util.Date;
14 import java.util.List;
15 import java.util.Map;
16 import java.util.UUID;
17
18 /**
19  * 课程执行与后续流程服务层，对应设计2.4 课程执行与后续流程模块所有接口的业务逻辑
20  * 涵盖签到、评价、反馈等功能，严格遵循权限校验和业务规则
21  */
22 @Service
23 public class CourseExecutionService {
24
25     @Autowired
26     private CourseCheckInMapper checkInMapper;
27     @Autowired
28     private CourseEvaluationMapper evaluationMapper;
29     @Autowired
30     private CourseFeedbackMapper feedbackMapper;
31     @Autowired
32     private ReservationOrderMapper orderMapper;
33     @Autowired
34     private ScheduleMapper scheduleMapper;
35     @Autowired
36     private CourseMapper courseMapper;
37
38     // 日期格式化工具，对应通用校验规则-时间格式
39     private static final SimpleDateFormat DATE_FORMAT = new
SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
40
41     /**
42      * 学生签到，对应设计2.4 签到功能
43      */
44     @Transactional
45     public void studentCheckIn(CourseCheckIn checkIn) {
46         // 1. 校验订单是否存在且已支付
47         ReservationOrder order =
orderMapper.selectOrderById(checkIn.getOrderId());
48         if (order == null) {
49             throw new ResourceNotFoundException("订单不存在，无法签到");
50         }
51         if (!"paid".equals(order.getOrderStatus())) {
52             throw new BusinessException("订单未支付，无法签到");
53         }
54
55         // 2. 校验排期是否存在
```

```

56         Schedule schedule =
scheduleMapper.selectScheduleById(checkIn.getScheduleId());
57         if (schedule == null) {
58             throw new ResourceNotFoundException("排期不存在, 无法签到");
59         }
60
61         // 3. 校验签到时间 (课程开始后30分钟内可签到, 对应设计2.4 签到规则)
62         try {
63             Date startTime = DATE_FORMAT.parse(schedule.getStartTime());
64             Date currentTime = new Date();
65             Date endCheckInTime = new Date(startTime.getTime() + 30 * 60 *
1000);
66             if (currentTime.before(startTime)) {
67                 throw new BusinessException("课程未开始, 无法签到");
68             }
69             if (currentTime.after(endCheckInTime)) {
70                 throw new BusinessException("签到时间已过, 无法签到");
71             }
72         } catch (ParseException e) {
73             throw new BusinessException("时间格式错误, 无法签到");
74         }
75
76         // 4. 校验是否已签到 (避免重复签到)
77         if (checkInMapper.selectByOrderAndStudent(checkIn.getOrderId(),
checkIn.getStudentId()) != null) {
78             throw new BusinessException("您已完成签到, 无需重复操作");
79         }
80
81         // 5. 补充教师ID (从课程中获取)
82         Course course = courseMapper.selectCourseById(order.getCourseId());
83         checkIn.setTeacherId(course.getTeacherId());
84
85         // 6. 生成签到ID, 设置签到状态和时间
86         String checkInId = UUID.randomUUID().toString();
87         checkIn.setCheckInId(checkInId);
88         checkIn.setCheckInStatus("checked");
89         checkIn.setCheckInTime(DATE_FORMAT.format(new Date()));
90
91         // 7. 插入签到记录到数据库
92         checkInMapper.insertCheckIn(checkIn);
93     }
94
95     /**
96      * 教师查询签到列表, 对应设计2.4 签到查询功能
97      */
98     public List<CourseCheckIn> getCheckInList(String scheduleId) {
99         // 校验排期是否存在

```

```
100         if (scheduleMapper.selectScheduleById(scheduleId) == null) {
101             throw new ResourceNotFoundException("排期不存在");
102         }
103         // 查询该排期下的所有签到记录
104         return checkInMapper.selectByScheduleId(scheduleId);
105     }
106
107     /**
108      * 学生提交课程评价, 对应设计2.4 评价功能
109      */
110     @Transactional
111     public Map<String, String> addEvaluation(CourseEvaluation evaluation) {
112         // 1. 校验订单是否存在且已支付、课程已结束
113         ReservationOrder order =
114 orderMapper.selectOrderById(evaluation.getOrderId());
115         if (order == null) {
116             throw new ResourceNotFoundException("订单不存在, 无法提交评价");
117         }
118         if (!"paid".equals(order.getOrderStatus())) {
119             throw new BusinessException("订单未支付, 无法提交评价");
120         }
121
122         // 2. 校验课程是否已结束 (仅课程结束后可评价, 对应设计2.4 评价规则)
123         Schedule schedule =
124 scheduleMapper.selectScheduleById(order.getScheduleId());
125         try {
126             Date endTime = DATE_FORMAT.parse(schedule.getEndTime());
127             Date currentTime = new Date();
128             if (currentTime.before(endTime)) {
129                 throw new BusinessException("课程未结束, 无法提交评价");
130             }
131         } catch (ParseException e) {
132             throw new BusinessException("时间格式错误, 无法提交评价");
133         }
134
135         // 3. 校验是否已提交评价 (避免重复评价)
136         if (evaluationMapper.selectByOrderAndStudent(evaluation.getOrderId(),
137 evaluation.getStudentId()) != null) {
138             throw new BusinessException("您已提交评价, 无需重复操作");
139         }
140
141         // 4. 校验评价分数 (1-5分, 对应设计2.4 评价规则)
142         if (evaluation.getScore() < 1 || evaluation.getScore() > 5) {
143             throw new BusinessException("评价分数需为1-5分");
144         }
145
146         // 5. 生成评价ID, 设置评价时间
```

```
144         String evaluationId = UUID.randomUUID().toString();
145         evaluation.setEvaluationId(evaluationId);
146         evaluation.setCreateTime(DateFormat.format(new Date()));
147
148         // 6. 插入评价到数据库
149         evaluationMapper.insertEvaluation(evaluation);
150
151         // 7. 组装返回数据
152         Map<String, String> resultMap = Map.of("evaluationId", evaluationId);
153         return resultMap;
154     }
155
156     /**
157      * 教师查询评价列表, 对应设计2.4 评价查询功能
158      */
159     public List<CourseEvaluation> getEvaluationList(String teacherId) {
160         // 查询该教师收到的所有评价
161         return evaluationMapper.selectByTeacherId(teacherId);
162     }
163
164     /**
165      * 提交课程反馈, 对应设计2.4 反馈功能
166      */
167     @Transactional
168     public Map<String, String> addFeedback(CourseFeedback feedback) {
169         // 1. 校验订单是否存在
170         ReservationOrder order =
orderMapper.selectOrderByid(feedback.getOrderid());
171         if (order == null) {
172             throw new ResourceNotFoundException("订单不存在, 无法提交反馈");
173         }
174
175         // 2. 校验提交人角色 (仅学生/教师可提交)
176         if (!"student".equals(feedback.getSubmitterRole()) &&
!"teacher".equals(feedback.getSubmitterRole())) {
177             throw new BusinessException("仅学生和教师可提交反馈");
178         }
179
180         // 3. 生成反馈ID, 设置默认状态和反馈时间
181         String feedbackId = UUID.randomUUID().toString();
182         feedback.setFeedbackId(feedbackId);
183         feedback.setHandleStatus("pending");
184         feedback.setCreateTime(DateFormat.format(new Date()));
185
186         // 4. 插入反馈到数据库
187         feedbackMapper.insertFeedback(feedback);
188     }
```

```
189         // 5. 组装返回数据
190         Map<String, String> resultMap = Map.of("feedbackId", feedbackId);
191         return resultMap;
192     }
193
194     /**
195      * 管理员处理反馈，对应设计2.4 反馈处理功能
196      */
197     @Transactional
198     public void handleFeedback(String feedbackId, String handleContent) {
199         // 1. 校验反馈是否存在
200         CourseFeedback feedback =
201             feedbackMapper.selectFeedbackById(feedbackId);
202         if (feedback == null) {
203             throw new ResourceNotFoundException("反馈不存在，无法处理");
204         }
205
206         // 2. 校验反馈状态（仅待处理反馈可处理）
207         if (!"pending".equals(feedback.getHandleStatus())) {
208             throw new BusinessException("反馈已处理，无需重复操作");
209         }
210
211         // 3. 更新反馈处理状态、内容和时间
212         feedback.setHandleStatus("handled");
213         feedback.setHandleContent(handleContent);
214         feedback.setHandleTime(DATE_FORMAT.format(new Date()));
215         feedbackMapper.updateFeedback(feedback);
216     }
217
218     /**
219      * 管理员查询反馈列表，对应设计2.4 反馈查询功能
220      */
221     public List<CourseFeedback> getFeedbackList(String handleStatus) {
222         // 按处理状态查询反馈（handleStatus为空则查询所有）
223         return feedbackMapper.selectFeedbackList(handleStatus);
224     }
225
226     /**
227      * 校验排期归属（辅助方法），对应设计2.4 权限控制
228      */
229     public void checkScheduleOwner(String scheduleId, String teacherId) {
230         Schedule schedule = scheduleMapper.selectScheduleById(scheduleId);
231         if (schedule == null) {
232             throw new ResourceNotFoundException("排期不存在");
233         }
234         Course course = courseMapper.selectCourseById(schedule.getCourseId());
235         if (!course.getTeacherId().equals(teacherId)) {
```


(注：文档部分内容可能由 AI 生成)